



AI Infra at Startups: The Real Challenges

Navigating the complex world of AI infrastructure with limited people, budget, and time while managing rapid change and the need to ship fast.

The Startup Reality Check

Small Teams

Everyone wears multiple hats across data, infrastructure, ML, and product development

Uncertain Requirements

High pivot risk makes planning difficult and long-term architecture decisions risky

Funding Constraints

Your runway directly dictates infrastructure choices and trade-offs

Remember: A "perfect" architecture $\neq$ a shipped product

Top Challenge #1: Cost Gravity

- **Expenses add up quickly:** GPUs, storage, data egress, and hosted APIs create significant costs
- **Hidden costs pile up:** Failed experiments, idle clusters, and flaky pipelines waste resources
- **Vendor traps:** Minimum commitments and lock-in penalties restrict flexibility



Top Challenge #2: Talent & Bandwidth



Talent Scarcity

Senior MLOps and SRE professionals are rare and expensive to hire

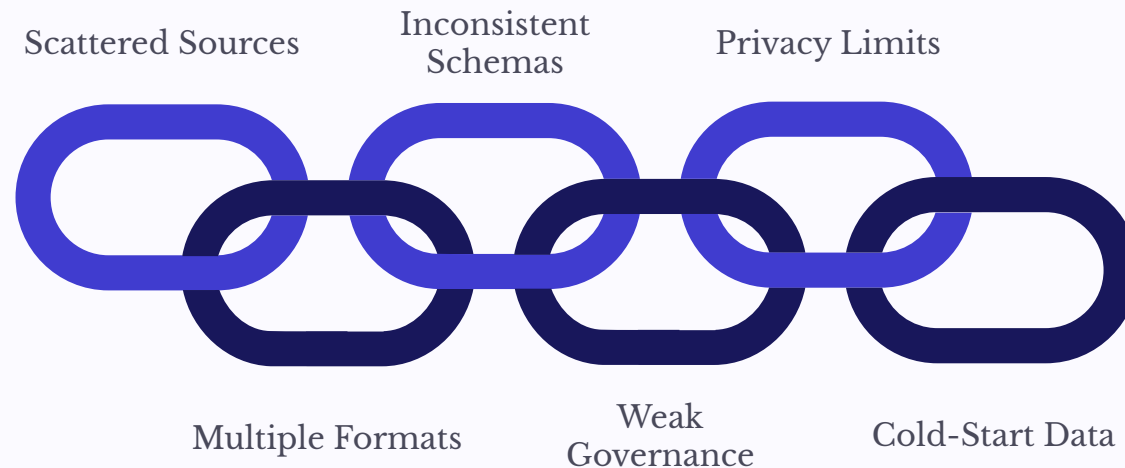
Context Switching

Toggling between research and infrastructure kills productivity and velocity

On-call Burnout

Managing training and serving incidents drains focus from core innovation

Top Challenge #3: Data Chaos



The data reality at most startups:

- Data scattered across multiple SaaS tools and storage buckets
- Inconsistent schemas and weak governance practices
- Cold-start problems: limited labeled data and privacy constraints

This chaos creates bottlenecks for model training, evaluation, and deployment—often becoming the invisible barrier to AI progress.

Top Challenge #4: Shipping vs Hardening

The Reality:

- Need demos [yesterday](#) → quick, hacky pipelines
- Reliability, security, and observability consistently lag behind
- Technical debt compounds rapidly, creating future bottlenecks



The delicate balance between shipping fast and building sustainable infrastructure



The Startup Trap: Ship fast but accumulate technical debt that eventually grinds progress to a halt

Top Challenge #5: Scale Uncertainty



Today

1 model in production

Predictable traffic

Single region



Tomorrow?

10+ models

A/B testing needs

Multi-region requirements



Unexpected

Traffic spikes from launches

PR mentions

Viral adoption

How do you build for now AND later with limited resources?

Symptom Checker (Self-Assessment)



Resource Bottlenecks

Do experiments constantly stall waiting for GPU availability?



Deployment Bottlenecks

Is a single person acting as the gatekeeper for all model deployments?



Cost Inefficiency

Are we consistently paying for idle hardware and unused resources?



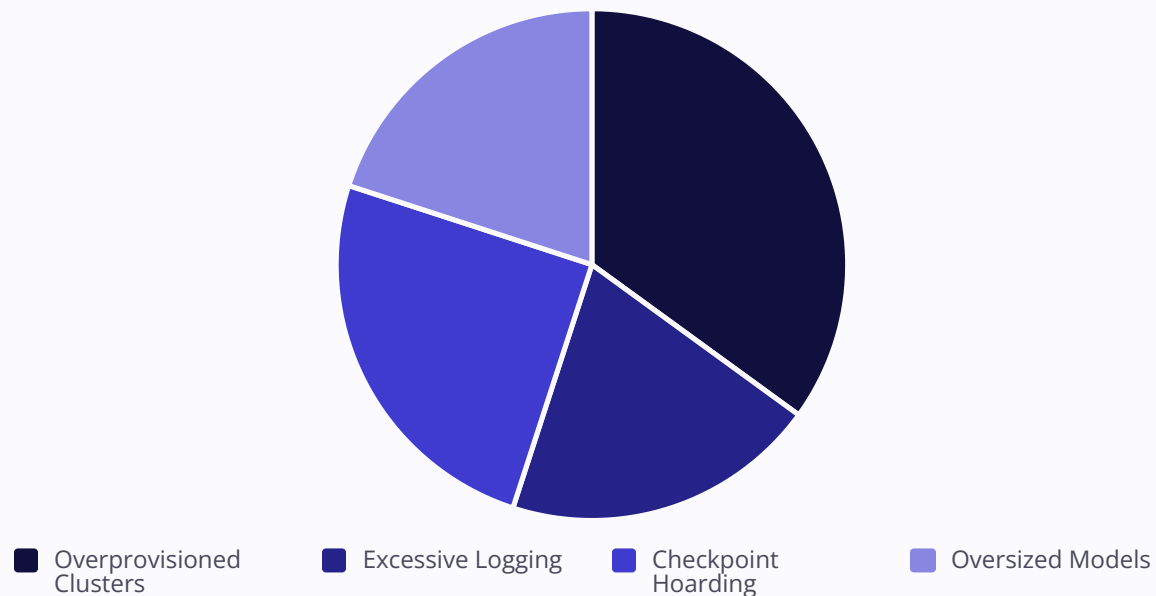
Reproducibility Issues

Can we reliably reproduce a model from 2 months ago if needed?



If you answered "yes" to any of these questions, your AI infrastructure needs attention.

Cost Pitfalls (Concrete Examples)



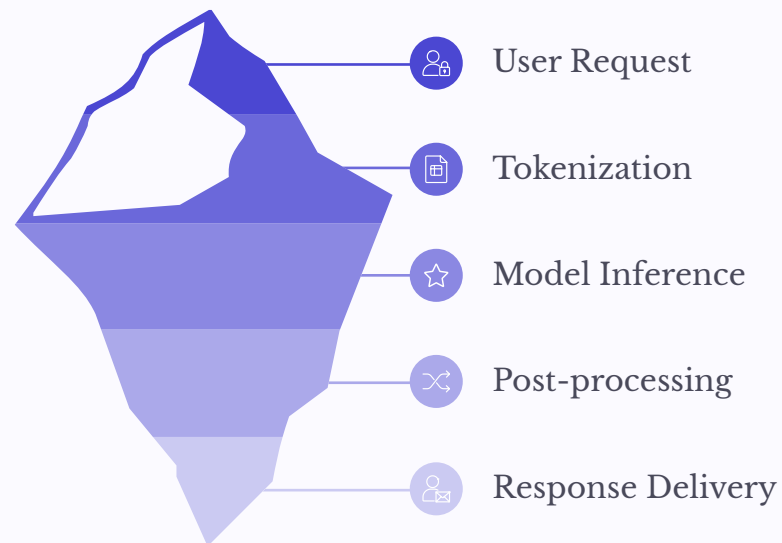
Common Wasteful Practices

- Overprovisioned clusters sitting idle between training runs
- Logging/metrics set at debug level 24/7 in production

Unnecessary Storage Costs

- Storing every checkpoint forever "just in case"
- Serving huge models for tiny performance gains

Latency & Throughput Pain



SLA Conflicts

End-user SLA expectations clash with batchy infrastructure realities

Processing Overhead

Tokenization and pre/post-processing bottlenecks on the hot path

Cold Start Problems

Serverless and container cold starts create inconsistent performance

Evaluation & Drift: Flying Blind



- **Missing Baselines**
Many teams ship without establishing proper baseline metrics
- **Metric Misalignment**
Offline evaluation metrics rarely align with online KPIs that matter
- **Unmonitored Drift**
Data drift and feedback loops go unmonitored until major failures occur

Security & Compliance (Early Stage Concerns)

Data Privacy Issues

PII regularly appears in logs, notebooks, and training datasets without proper controls

Poor Credential Management

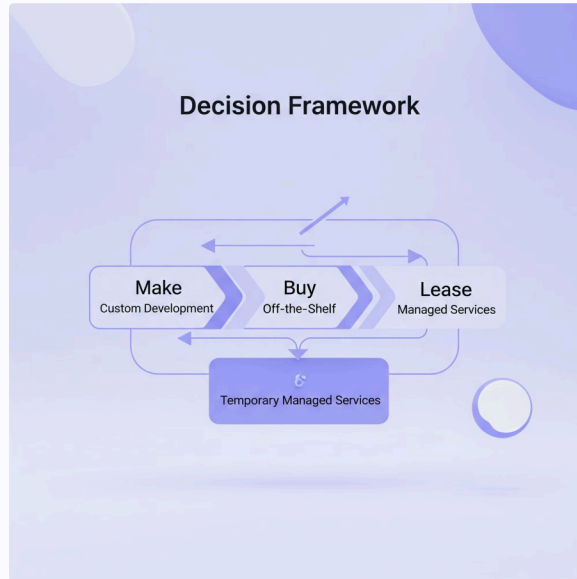
Shared API keys and long-lived credentials create unnecessary security risks

Enterprise Sales Blockers

Lack of RBAC and audit trails becomes a major blocker for enterprise deals

These issues are often ignored until they block a major customer deal

Make/Buy/Lease Framework



MAKE

Build custom when it's your core differentiator and directly impacts product quality

BUY

Purchase off-the-shelf for commodity needs (monitoring, CI/CD, feature stores)

LEASE

Use managed GPUs and LLM APIs while learning your demand patterns

Pro Tip: Revisit quarterly and renegotiate as you scale

Survival Architecture: MVP → PMF

Phase 0: Scrappy Experimentation

- Single repository, one environment
- Hosted LLM APIs where possible
- Minimal infrastructure overhead

Phase 2: Scaling Operations

- Autoscaled model serving
- Evaluation harnesses
- Cost dashboards and alerts

Phase 1: Basic Productionization

- Simple pipelines (Airflow/Prefect)
- Centralized object storage
- Primitive model registry

Phase 3: Enterprise Ready

- Multi-tenant architecture
- RBAC and audit logging
- Comprehensive observability
- Canary deployments

Lean GPU Strategy

1 Use Spot/Preemptible Instances

For training workloads with checkpoint resume capabilities

2 Right-size Your Compute

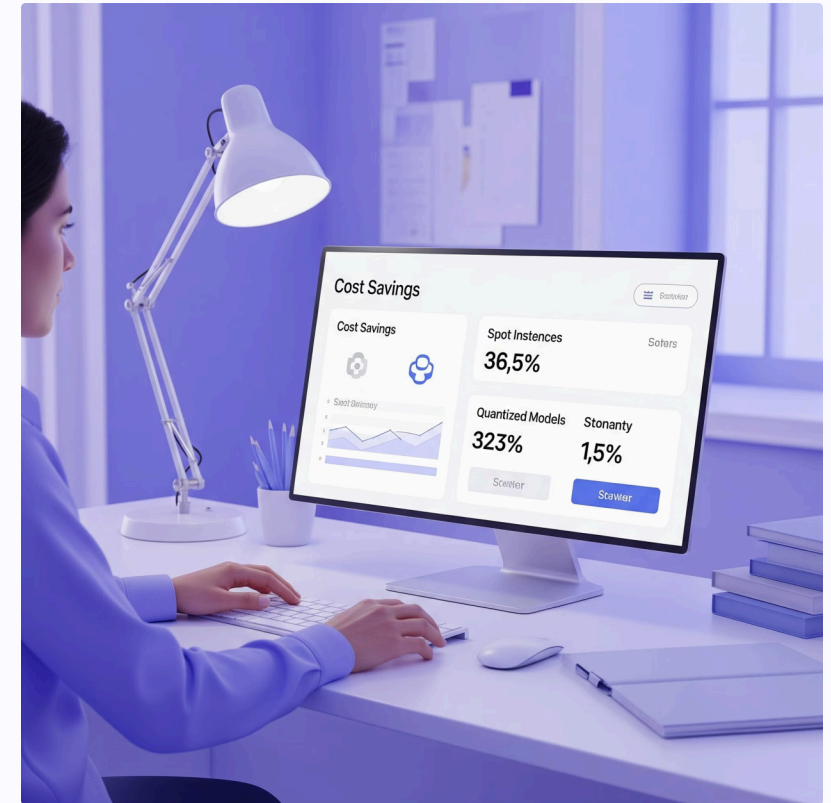
Implement AMP/quantization to maximize efficiency

3 Prioritize High-ROI Experiments

Queue jobs strategically, not on a first-come basis

4 Consider CPU Inference

Explore CPU/INT8 options for smaller or edge models



Optimized GPU utilization can cut costs by 60-70%

Data Playbook for AI Startups

1

Centralize Storage

Create a central object store with clear schema definitions and versioning

2

Create Golden Datasets

Develop small, high-quality datasets that enable fast iteration cycles

3

Build Labeling Loops

Implement continuous labeling and weak supervision while tracking data provenance

4

Establish Privacy Guardrails

Develop early privacy posture with data masking and retention policies

📌 Data strategy often matters more than model architecture for early-stage AI products

Evaluation-First Culture

Why It Matters

Without proper evaluation, you're flying blind and risking both product quality and unnecessary spending on marginal improvements.



Implementation Steps

1. **Define north-star product metrics** (CTR, CSAT, conversion) that directly tie to business value
2. **Build lightweight evaluation harnesses** with regression tests for core functionality
3. **Deploy changes gradually** using shadow/canary testing before full rollout
4. **Track performance vs. cost** per 1,000 inferences to optimize ROI

Reliability Without the Bloat



Start Simple

Begin with single-region deployments and add high availability only when needed and justified



Basic SLOs

Define simple service level objectives for latency and availability with clear error budgets



Minimal Observability

Implement structured logs and targeted tracing, with alerts focused on user impact



Simple Playbooks

Create straightforward runbooks for common scenarios like rollbacks and hotfixes

You don't need Google-scale SRE practices for an early-stage startup

Security Basics That Win Deals

Access Control

- RBAC by default for all systems
- Eliminate shared secrets and credentials

Data Protection

- Tenant-scoped storage buckets
- Comprehensive audit logging

Compliance Fast-Track

- PII handling guidelines and log scrubbing
- "SOC2-lite" roadmap for enterprise deals

These basics will satisfy 80% of enterprise security reviews without massive overhead



Avoiding Lock-In (Pragmatic Approach)



Keep Artifacts Portable

Use standard formats like ONNX/GGUF and Hugging Face to maintain flexibility



Abstract at Boundaries

Create thin abstractions at provider boundaries, not deep in your codebase



Regular Data Exports

Establish routine exports of datasets and checkpoints to prevent silent lock-in



Contract Protections

Negotiate favorable egress fees and capacity guarantees in vendor contracts

People & Process: The Human Element



1 Rotate Responsibilities

Implement "infra captain" rotation to eliminate single points of failure in your team

2 Regular Reviews

Hold concise 15-minute weekly cost and reliability reviews to catch issues early

3 Light Postmortems

Make incident postmortems lightweight but mandatory to build institutional knowledge

4 Documentation-as-Code

Keep docs in your repo with one-command reproduction for training processes

Budget Guardrails



Set Clear Limits

Establish monthly GPU budgets with daily burn rate alerts to prevent surprises



Project Quotas

Implement quotas per project and automatically stop idle jobs after threshold periods



Track Experiment Costs

Assign cost tracking to each feature flag and experiment for ROI calculation



Tie Spend to Outcomes

Connect infrastructure spending directly to business outcomes like win rates and retention

AI costs can spiral quickly without these guardrails in place

Tooling Starter Pack (Lean Edition)



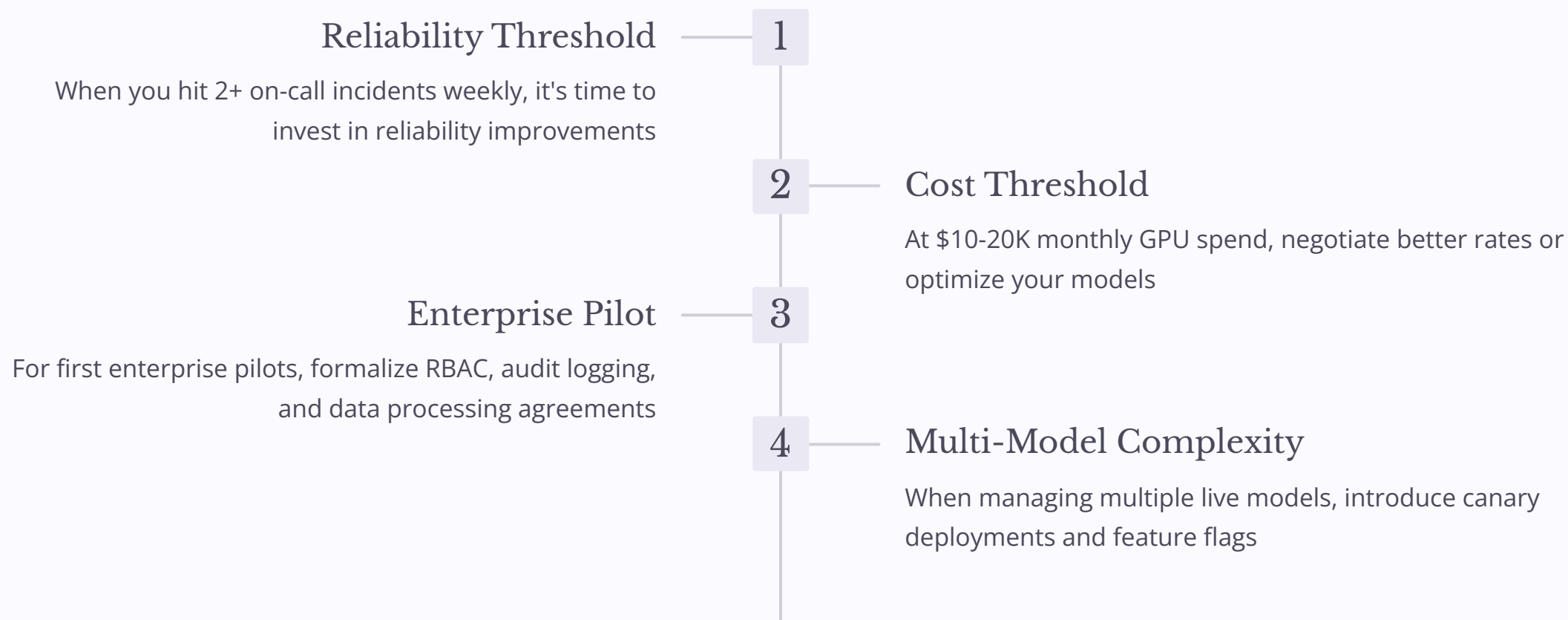
High ROI Tooling Categories

- **Orchestration:** Prefect or Airflow Lite for workflow management
- **Tracking:** MLflow for experiment and model registry needs

Essential Infrastructure

- **Serving:** Triton or FastAPI with autoscaling capabilities
- **Observability:** Prometheus + Grafana stack, with Sentry for errors
- **Cost Management:** Kubecost or native cloud billing tools

When to Uplevel Your Infrastructure



Don't overengineer early, but don't wait for a crisis to upgrade

Case Study Template (Use in Class)



Context

Product description, target users, and key performance indicators

Constraints

Team size, funding runway, and required service level agreements

Decisions

Make/buy/lease choices, model size and precision trade-offs

Outcomes

Resulting cost structure, latency, accuracy, and iteration speed

Use this template to work through your own AI infrastructure decisions

Summary: Pragmatic AI Infrastructure



The Startup AI Balancing Act

Startups must constantly juggle competing priorities:

- **Cost efficiency** vs. compute power
- **Development speed** vs. reliability
- **Data access** vs. privacy/security

The Winning Approach

Favor **simple, portable, cost-aware** infrastructure that enables rapid iteration. Only pay the complexity tax when clearly justified by business needs.